

Dependency Injection

Em Spring

Inversion of Control

- Normalmente o vosso programa chama métodos numa biblioteca. Como o utilizador da biblioteca, vocês têm o controlo do fluxo do programa. A biblioteca apenas ajuda em alguns passos específicos.
 - Classe A chama um método na classe B, que chama um método na classe C.
- **Inversion of Control** (IoC) muda este paradigma. Agora, as frameworks é que definem o flow do programa, e chama o vosso código apenas quando decide que precisa.
 - Temos código por callbacks:
 - Quando a aplicação arrancar, chama o método *init*
 - ...
 - Antes da aplicação fechar, chama o método *dispose*

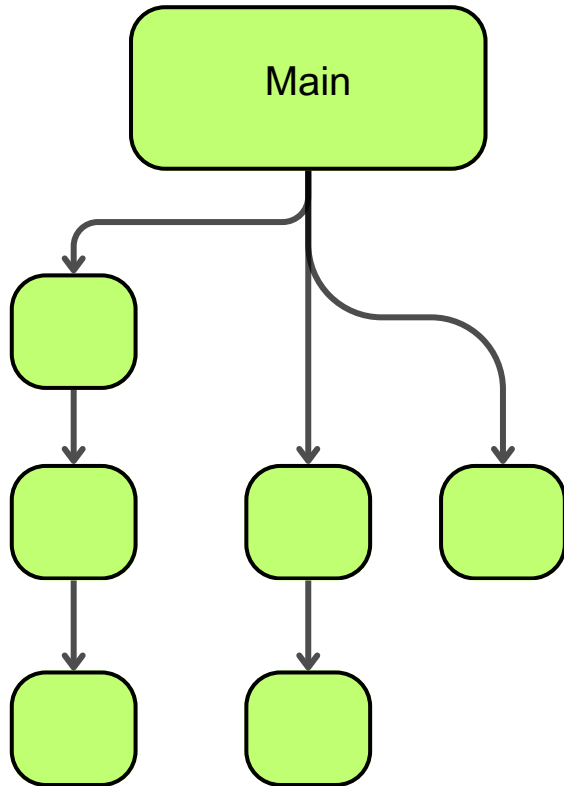
Inversion of Control

Exemplos

- **GUI Frameworks.** A framework trata do main-loop, e chama o vosso código que trata de eventos.
- **Web Frameworks.** A framework é responsável por tratar do servidor HTTP e apenas chama o vosso código específico para a página que está a ser pedida. O fluxo é gerido pela framework.

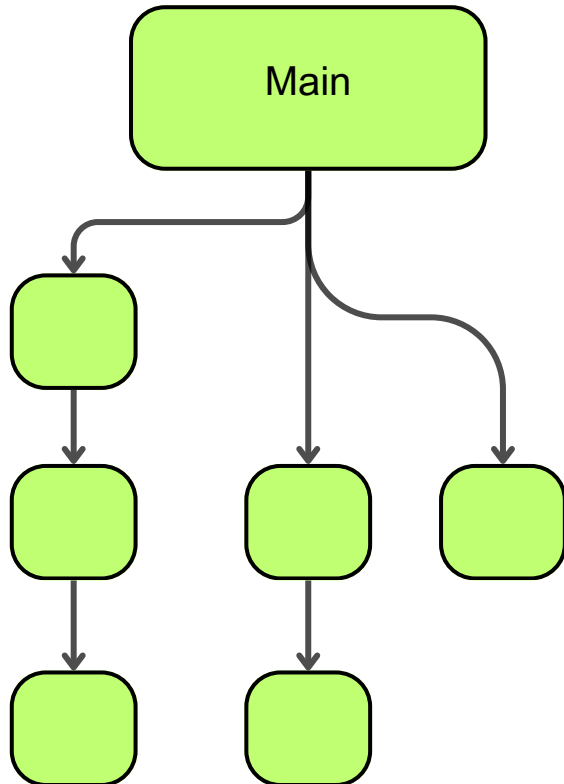
Dependency Injection

Modelo Tradicional

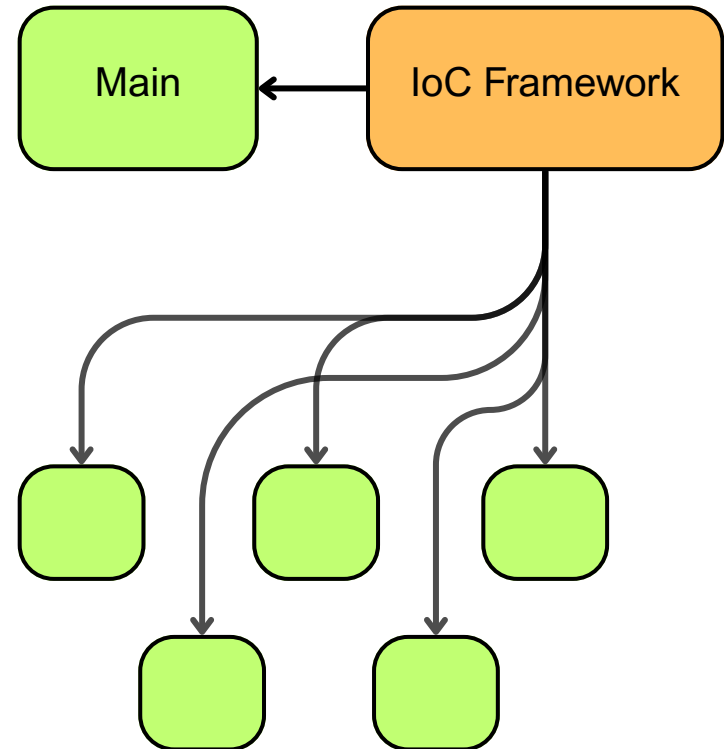


Dependency Injection

Modelo Tradicional



Injeção de Dependência



Dependency Injection

Uma forma de Inversion of Control

- A ideia principal é poder definir apenas uma interface que precisamos no nosso código, e ter um objecto (container) que preenche a implementação dessa interface.

```
public interface PersonRepository extends Repository<Person, Long> { ... }
```

Dependency Injection

Uma forma de Inversion of Control

- A ideia principal é poder definir apenas uma interface que precisamos no nosso código, e ter um objecto (container) que preenche a implementação dessa interface.

```
public interface PersonRepository extends Repository<Person, Long> { ... }
```

- Neste caso, o Spring vai criar uma implementação de PersonRepository automaticamente, e injectar onde a pedirmos.

```
@Autowired
```

```
private PersonRepository repository;
```

- O código que usa o repositório não tem de conhecer a implementação! É injectada!

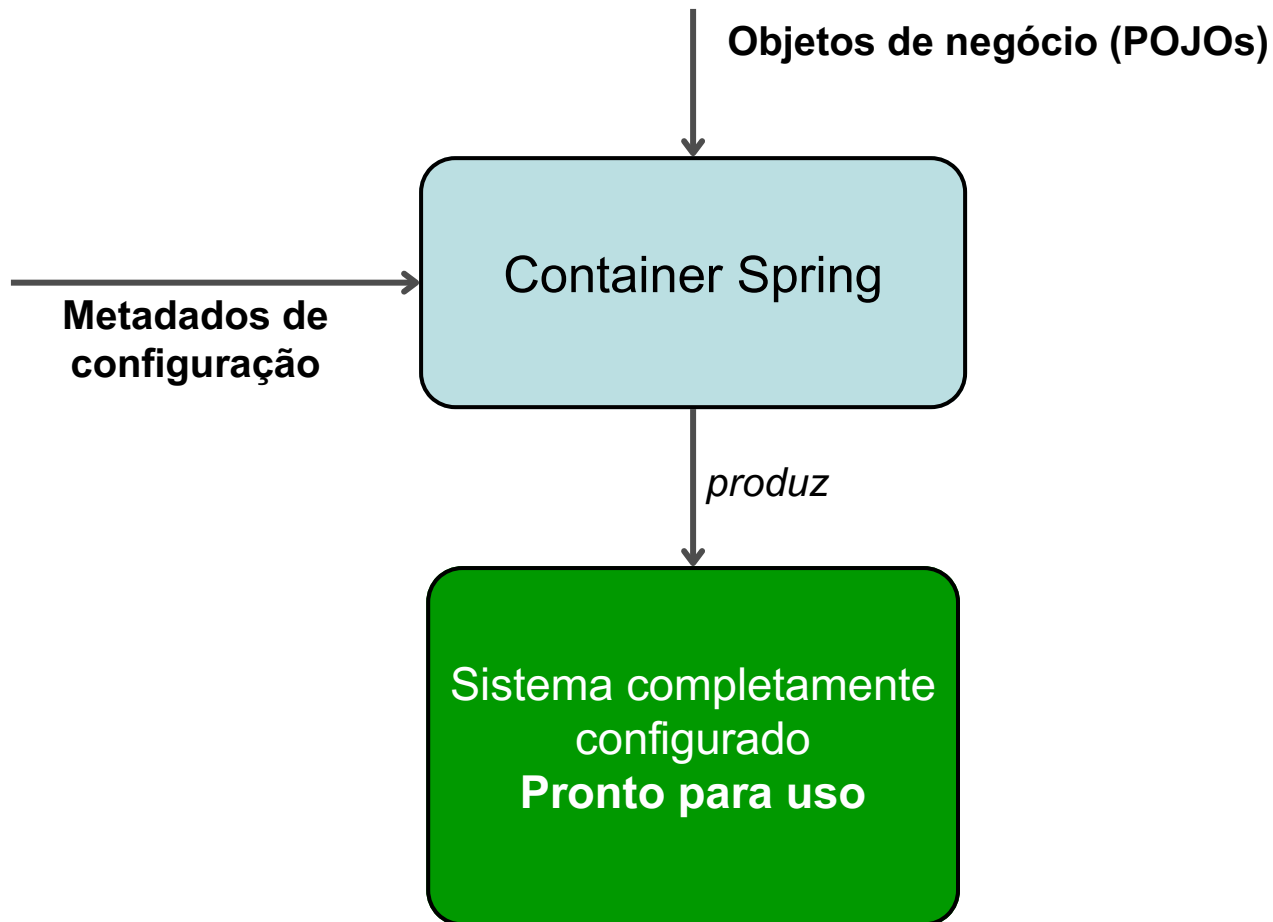
Dependency Injection

IoC container

- Um container é responsável por gerir um conjunto de objectos (beans), incluindo criar e preencher as suas dependências.
- As escolhas definidas num container são propagadas para os beans criados por este.
- No Spring
 - `org.springframework.context.ApplicationContext`
 - `org.springframework.beans.factory.BeanFactory`

Dependency Injection

IoC container



Dependency Injection

IoC container

- Podemos definir metadados a injetar através de XML, ou anotações em Java.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">
  <!-- services -->
  <bean id="petStore"
        class="org.springframework.samples.jpetsy.store.services.PetStoreServiceImpl">
    <property name="accountDao" ref="accountDao"/>
    <property name="itemDao" ref="itemDao"/>
    <!-- additional collaborators and configuration for this bean go here -->
  </bean>
  <!-- more bean definitions for services go here -->
</beans>
```

Dependency Injection

IoC

- Vantagens:
 - Maior decoupling de componentes
 - Mais fácil de testar código (usando mocks para substituir componentes)
- Desvantagens
 - Maior complexidade na injeção
 - O flow da execução não é transparente, nem fácil de perceber
 - Podem ter perdas de performance, pelo uso de reflection

Beans

- Um Bean é uma classe que é criada e gerida por um IoC container.

```
@Component
public class Company {
    private Address address;

    public Company(Address address) {
        this.address = address;
    }

    // getter, setter and other properties
}
```

Beans

- Um Bean é uma classe que é criada e gerida por um IoC container.

```
@Component
public class Company {
    private Address address;
    public Company(Address address) {
        this.address = address;
    }

    // getter, setter and other properties
}
```

```
@Configuration
@ComponentScan(basePackageClasses = Company.class)
public class Config {
    @Bean
    public Address getAddress() {
        return new Address("High Street", 1000);
    }
}
```

Beans

- Um Bean é uma classe que é criada e gerida por um IoC container.

```
@Component
public class Company {
    private Address address;
    public Company(Address address) {
        this.address = address;
    }
    // getter, setter and other properties
}
```

```
@Configuration
@ComponentScan(basePackageClasses = Company.class)
public class Config {
    @Bean
    public Address getAddress() {
        return new Address("High Street", 1000);
    }
}
```

```
ApplicationContext context = new AnnotationConfigApplicationContext(Config.class);
Company company = context.getBean("company", Company.class);
assertEquals("High Street", company.getAddress().getStreet());
```

Beans

Lifecycle

- Se precisarem de fazer alguma limpeza ao objecto...

```
public class CachingMovieLister {  
  
    @PostConstruct   
    public void populateMovieCache() {  
        // populates the movie cache upon initialization...  
    }  
  
    @PreDestroy   
    public void clearMovieCache() {  
        // clears the movie cache upon destruction...  
    }  
}
```

Como usar Beans

Preencher beans

```
public class SimpleMovieLister {  
  
    private MovieFinder movieFinder;  
  
    @Required  
    public void setMovieFinder(MovieFinder movieFinder) {  
        this.movieFinder = movieFinder;  
    }  
  
    // ...  
}
```

- @Required num setter indica que precisamos de injectar um MovieFinder nesta classe.
- (Ajuda a evitar NullPointerExceptions)

Como usar Beans

Preencher beans

```
public class MovieRecommender {  
  
    @Autowired  
    private MovieCatalog movieCatalog;  
  
    private CustomerPreferenceDao customerPreferenceDao;  
  
    @Autowired  
    public MovieRecommender(CustomerPreferenceDao customerPreferenceDao) {  
        this.customerPreferenceDao = customerPreferenceDao;  
    }  
  
    @Autowired  
    public void prepare(MovieCatalog movieCatalog,  
                        CustomerPreferenceDao customerPreferenceDao) {  
        this.movieCatalog = movieCatalog;  
        this.customerPreferenceDao = customerPreferenceDao;  
    }  
}
```

- O @Autowired pode ser usado para preencher argumentos em métodos, construtores ou atributos.

Como usar Beans

Preencher beans

```
public class ClientController {  
  
    @Autowired  
    private CustomerServiceClient customerService;  
  
    // ...  
}
```

```
public interface CustomerServiceClient {  
  
    List<CustomerDto> getAllCustomers();  
  
    CustomerDto getCustomerByVat(String vat);  
  
}
```

```
@Service  
public class CustomerServiceClientImpl  
implements CustomerServiceClient{  
  
    // ...  
  
}
```

- Se só houver uma implementação de uma interface no contexto ela será usada (caso contrário → **@Primary**)

@Autowired

Onde usar?

- Para projetos mais complexos, devemos reservar o uso do *@Autowired* para **métodos e construtores**
 - Mais fácil de testar (dependências não ficam escondidas dentro da classe)
 - Injetando as dependências no construtor, é possível declarar os atributos como *final* (+segurança)
 - Deixa claro, logo na assinatura do construtor, quais objetos são necessários para o funcionamento da classe.
- Se a classe só tiver um construtor, é assumido que esse construtor faz uso do *@Autowired*

@Autowired

E nos atributos?

- Se todos os atributos forem definidos com *@Autowired* não precisamos de um construtor, nem getters ou setters
 - Código mais direto e com menos boilerplate
 - Implementação mais ágil
 - Mais intuitivo e menos complexo do que gerenciar múltiplos construtores
- **TL;DR:** Para projetos menores ou para versões iniciais, utilizar *@Autowired* nos atributos não é mal. Conforme o projeto fica mais maduro, deve-se migrar para utilização somente nos construtores / métodos

Como usar Beans

Como criar beans

Annotation	Descrição
@Component	Provider padrão de beans.
@Repository	Especialização do @Component para repositório da camada de acesso aos dados (Data Access Object)
@Service	Especialização do @Component para fornecer serviços (Camada de Serviços)
@Controller	Especialização do @Component para controladores (Controller do MVC)

Como usar Beans

Como criar beans

Annotation	Descrição
<code>@Component</code>	Provider padrão de beans.
<code>@Repository</code>	Especialização do <code>@Component</code> para repositório da camada de acesso aos dados (Data Access Object)
<code>@Service</code>	Especialização do <code>@Component</code> para fornecer serviços (Camada de Serviços)
<code>@Controller</code>	Especialização do <code>@Component</code> para controladores (Controller do MVC)

<code>@RestController</code>	Combinação de <code>@Controller</code> com <code>@ResponseBody</code>
------------------------------	---

Beans

Scopes

- Uma definição de um bean é uma receita de como criar beans desse “tipo”. Quando é que são criados de facto, depende do seu scope.

Only in WebApplications	Scope	Descrição
	singleton	(Default) Uma única definição de bean para uma única instância de objeto por Spring IoC container.
	prototype	Uma única definição de bean para qualquer número de instâncias de objeto.
	request	Uma única definição de bean para o ciclo de vida de uma única requisição HTTP; isto é, cada requisição HTTP possui sua própria instância de bean criada a partir de uma única definição de bean.
	session	Uma única definição de bean para o ciclo de vida de uma HTTP <i>Session</i> .
	global session	Uma única definição de bean para o ciclo de vida de uma HTTP <i>Session</i> global. Geralmente, válido somente quando utilizado em um contexto de portlet.

Como usar Beans

Criar beans com diferentes scopes

```
@Component
public class FactoryMethodComponent {

    private static int i;

    @Bean @Qualifier("public")
    public TestBean publicInstance() {
        return new TestBean("publicInstance");
    }
}
```


Como usar Beans

Criar beans com diferentes scopes

```
@Component
public class FactoryMethodComponent {

    private static int i;

    @Bean @Qualifier("public")
    public TestBean publicInstance() {
        return new TestBean("publicInstance");
    }
}
```



Como usar Beans

Criar beans com diferentes scopes

@Component

public class FactoryMethodComponent {

private static int i;

@Bean @Qualifier("public")

public TestBean publicInstance() {

return new TestBean("publicInstance");

}

Como usar Beans

Criar beans com diferentes scopes

@Component

```
public class FactoryMethodComponent {
```

```
    private static int i;
```

@Bean @Qualifier("public")

```
public TestBean publicInstance() {  
    return new TestBean("publicInstance");  
}
```

// use of a custom qualifier and autowiring of method parameters

@Bean

```
protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,  
                                     @Value("#{privateInstance.age}") String country) {  
    TestBean tb = new TestBean("protectedInstance", 1);  
    tb.setSpouse(spouse);  
    tb.setCountry(country);  
    return tb;  
}
```

Como usar Beans

Criar beans com diferentes scopes

@Component

```
public class FactoryMethodComponent {
```

```
    private static int i;
```

@Bean @Qualifier("public")

```
public TestBean publicInstance() {  
    return new TestBean("publicInstance");  
}
```

// use of a custom qualifier and autowiring of method parameters

@Bean

```
protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,  
                                     @Value("#{privateInstance.age}") String country) {  
    TestBean tb = new TestBean("protectedInstance", 1);  
    tb.setSpouse(spouse);  
    tb.setCountry(country);  
    return tb;  
}
```



Como usar Beans

Criar beans com diferentes scopes

@Component

```
public class FactoryMethodComponent {
```

```
    private static int i;
```

@Bean @Qualifier("public")

```
public TestBean publicInstance() {  
    return new TestBean("publicInstance");  
}
```

// use of a custom qualifier and autowiring of method parameters

@Bean

```
protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,  
                                     @Value("#{privateInstance.age}") String country) {  
    TestBean tb = new TestBean("protectedInstance", 1);  
    tb.setSpouse(spouse);  
    tb.setCountry(country);  
    return tb;  
}
```

@Bean @Scope(BeanDefinition.SCOPE_SINGLETON)

```
private TestBean privateInstance() {  
    return new TestBean("privateInstance", i++);  
}
```

Como usar Beans

Criar beans com diferentes scopes

```
@Component
public class FactoryMethodComponent {

    private static int i;

    @Bean @Qualifier("public")
    public TestBean publicInstance() {
        return new TestBean("publicInstance");
    }

    // use of a custom qualifier and autowiring of method parameters

    @Bean
    protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,
                                          @Value("#{privateInstance.age}") String country) {
        TestBean tb = new TestBean("protectedInstance", 1);
        tb.setSpouse(spouse);
        tb.setCountry(country);
        return tb;
    }

    @Bean @Scope(BeansDefinition.SCOPE_SINGLETON)
    private TestBean privateInstance() {
        return new TestBean("privateInstance", i++);
    }
}
```



Como usar Beans

Criar beans com diferentes scopes

```
@Component
public class FactoryMethodComponent {

    private static int i;

    @Bean @Qualifier("public")
    public TestBean publicInstance() {
        return new TestBean("publicInstance");
    }

    // use of a custom qualifier and autowiring of method parameters

    @Bean
    protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,
                                           @Value("#{privateInstance.age}") String country) {
        TestBean tb = new TestBean("protectedInstance", 1);
        tb.setSpouse(spouse);
        tb.setCountry(country);
        return tb;
    }

    @Bean @Scope(BeanDefinition.SCOPE_SINGLETON)
    private TestBean privateInstance() {
        return new TestBean("privateInstance", i++);
    }

    @Bean @Scope(value = WebApplicationContext.SCOPE_SESSION,
                  proxyMode = ScopedProxyMode.TARGET_CLASS)
    public TestBean requestScopedInstance() {
        return new TestBean("requestScopedInstance", 3);
    }
}
```

Como usar Beans

Criar beans com diferentes scopes

```
@Component
public class FactoryMethodComponent {

    private static int i;

    @Bean @Qualifier("public")
    public TestBean publicInstance() {
        return new TestBean("publicInstance");
    }

    // use of a custom qualifier and autowiring of method parameters

    @Bean
    protected TestBean protectedInstance(@Qualifier("public") TestBean spouse,
                                          @Value("#{privateInstance.age}") String country) {
        TestBean tb = new TestBean("protectedInstance", 1);
        tb.setSpouse(spouse);
        tb.setCountry(country);
        return tb;
    }

    @Bean @Scope(BeanDefinition.SCOPE_SINGLETON)
    private TestBean privateInstance() {
        return new TestBean("privateInstance", i++);
    }

    @Bean @Scope(value = WebApplicationContext.SCOPE_SESSION,
                  proxyMode = ScopedProxyMode.TARGET_CLASS)
    public TestBean requestScopedInstance() {
        return new TestBean("requestScopedInstance", 3);
    }
}
```



Como usar Beans

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Component;

@Component
public class BeanUser {

    @Autowired
    @Qualifier("public") // Injects the bean from publicInstance()
    private TestBean publicBean;

    @Autowired
    private TestBean protectedInstance; // Automatically matches by name

    @Autowired
    private TestBean privateInstance; // Same: matched by method name

    @Autowired
    private TestBean requestScopedInstance; // Session-scoped bean (proxied)

    public void showBeans() {
        System.out.println("Public Bean: " + publicBean.getName());
        System.out.println("Protected Bean: " + protectedInstance.getName());
        System.out.println("Private Bean: " + privateInstance.getName());
        System.out.println("Request Scoped Bean: " + requestScopedInstance.getName());
    }
}
```

Beans

Scopes

- Beans de sessão podem ser usados para dados que precisam de ser acumulados entre pedidos, durante uma transacção de negócio, mas ainda não estão prontos para serem persistidos.
 - Ex: o conteúdo do carrinho de compras.
- O ideal é evitar guardar informação entre pedidos, mas tal pode não ser possível.
 - Exemplo 1: No Salesys, criar a venda, e passar em cada pedido um identificador da venda.
 - Exemplo 2: Os links numa página web conterem os identificadores necessários para não ter de guardar o estado do lado do servidor. (Restful!)

Repositórios

```
public interface CrudRepository<T, ID extends Serializable>
    extends Repository<T, ID> {

    <S extends T> S save(S entity);

    T findOne(ID primaryKey);

    Iterable<T> findAll();
    Long count();

    void delete(T entity);

    boolean exists(ID primaryKey);

    // ... more functionality omitted.
}
```

- Extender o Repository faz com que o Spring crie o código-cola com o JPA automaticamente!

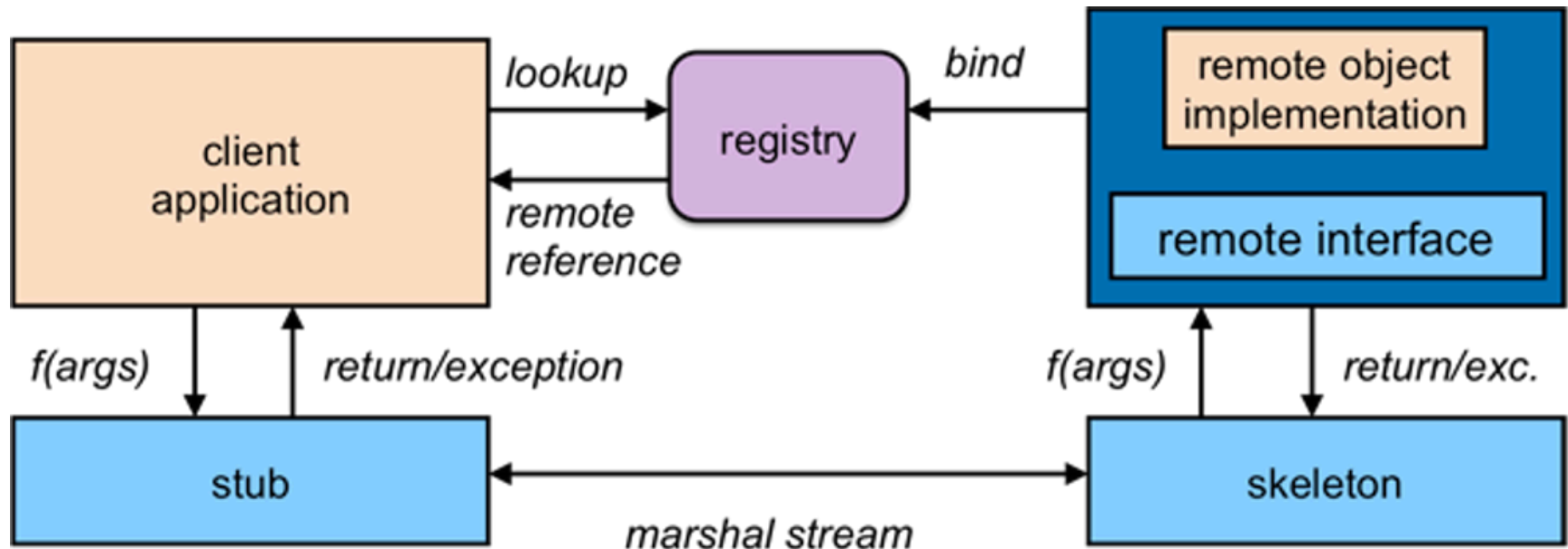
Repositórios

```
public interface PersonRepository extends Repository<User, Long> {  
  
    List<Person> findByEmailAddressAndLastname(EmailAddress emailAddress, String lastname);  
  
    // Enables the distinct flag for the query  
    List<Person> findDistinctPeopleByLastnameOrFirstname(String lastname, String firstname);  
    List<Person> findPeopleDistinctByLastnameOrFirstname(String lastname, String firstname);  
  
    // Enabling ignoring case for an individual property  
    List<Person> findByLastnameIgnoreCase(String lastname);  
  
    // Enabling ignoring case for all suitable properties  
    List<Person> findByLastnameAndFirstnameAllIgnoreCase(String lastname, String firstname);  
  
    // Enabling static ORDER BY for a query  
    List<Person> findByLastnameOrderByFirstnameAsc(String lastname);  
    List<Person> findByLastnameOrderByFirstnameDesc(String lastname);  
}
```

- Podem construir queries apenas pelo nome do método.

Remote Method Interface (RMI)

- O RMI é um mecanismo que permite a uma JVM chamar um método que corre noutra JVM, através de sockets TCP.



Exportar RMI Services

```
public interface CabBookingService {  
    Booking bookRide(String pickUpLocation) throws BookingException;  
}
```

Servidor

```
@Bean  
CabBookingService bookingService() {  
    return new CabBookingServiceImpl();  
}  
  
@Bean  
RmiServiceExporter exporter(CabBookingService implementation) {  
    Class<CabBookingService> serviceInterface =  
        CabBookingService.class;  
    RmiServiceExporter exporter = new RmiServiceExporter();  
  
    exporter.setServiceInterface(serviceInterface);  
    exporter.setService(implementation);  
    exporter.setServiceName(serviceInterface.getSimpleName());  
    exporter.setRegistryPort(1099);  
  
    return exporter;  
}
```

Cliente

```
@Bean  
RmiProxyFactoryBean cabBookingService() {  
    RmiProxyFactoryBean proxy =  
        new RmiProxyFactoryBean();  
  
    proxy.setServiceUrl("rmi://localhost:1099/CabBookingService");  
    proxy.setServiceInterface(CabBookingService.class);  
    return proxy;  
}
```

Remote Method Interface (RMI)

- As chamadas a métodos RMI são mais lentas do que chamadas locais (estabelecer socket + serialização + trânsito na rede).
- Os argumentos precisam de ser serializáveis para disco (File não é serializável...)
- **Padrão Remote Facade:**
 - Tentamos ter uma api de grão mais grosso (objectos maiores, que escondem comportamentos complexos) de forma a minimizar o overhead de chamadas externas.
 - Assim com um método fazemos o que numa API local faríamos em 10 ou 20.

Dependency Injection

Em Spring